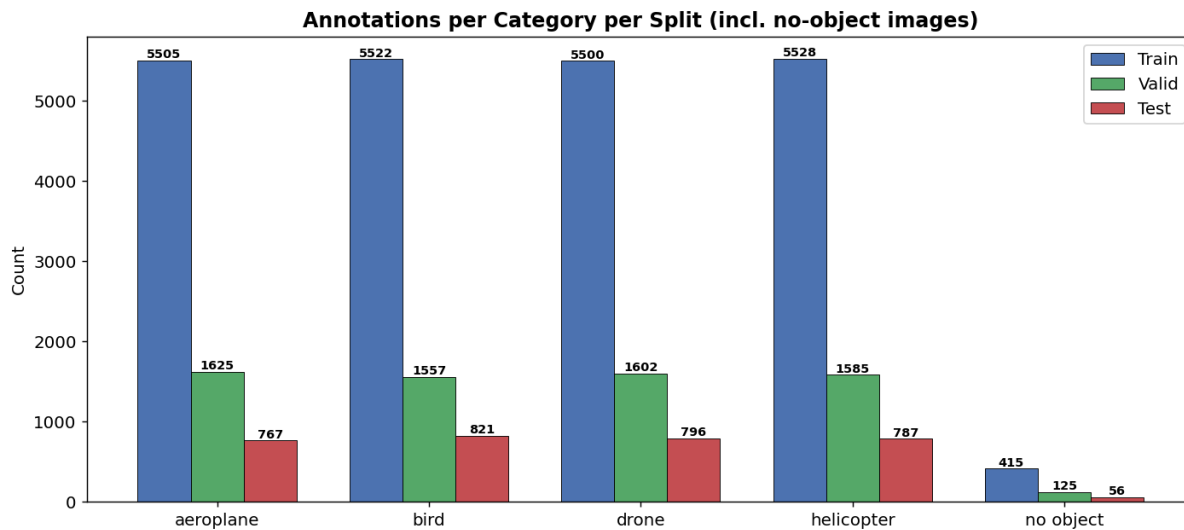


Airborne Object Detection (a1989895)

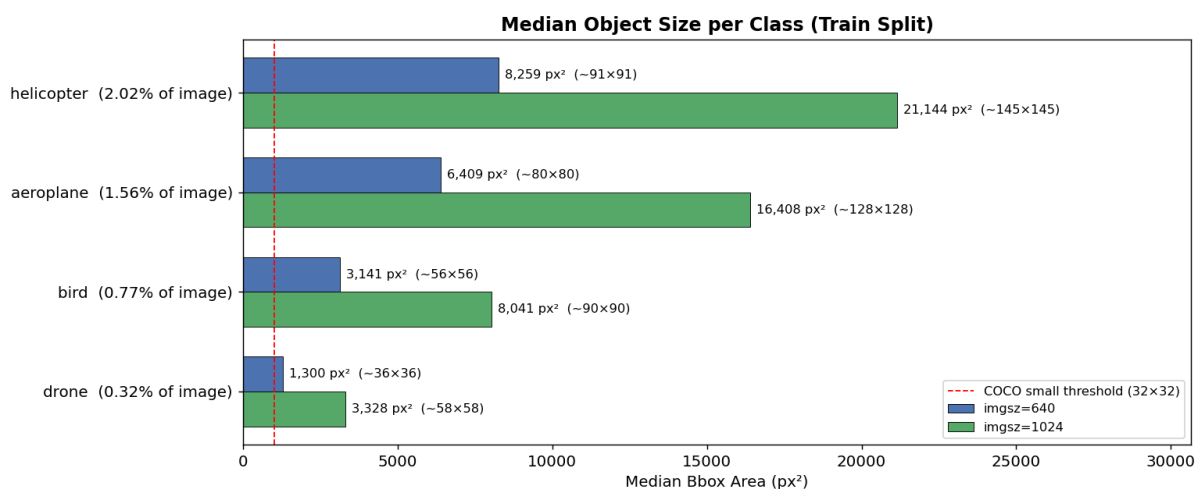
1. Data Visualization:

Dataset link: <https://data.mendeley.com/datasets/cd5z895tr2/1>

Training Data contains 15.7K images.



This is a balanced dataset because annotations are balanced in all 3 splits. It doesn't matter how many images we have in each split. Because an image may contain more than one annotation.



Reading the data of drone from above graph: drone images generally occupy 0.32% of the entire image. At 640px input image size a drone occupies 1300 px² which is like 36x36 pixels (which is just above the Small object threshold of COCO model). While at 1024px input image size drone occupies 3328 px² which is like 58x58 pixels.

More the number of pixels occupied by an object in the image, easier for the model to detect it, better the accuracy of the model. We need to balance this with batch size during training as the bottleneck becomes the Memory of the computer that's training the model.

2. Data Quality checks:

What to search for ?

- Verify if each corresponding image has a corresponding text file of annotation (class and bounding box coordinates) and vice versa.

- b. Verify if the format of annotations is correct in each .txt file.

Issues found :

In yolov8 annotation format (which is compatible for yolo26) : our dataset had 3 label files with polygon/segmentation-style annotations (21-27 values per line) instead of YOLO bounding box format (5 values per line). Solution : Since there were only 3 such files we resolved to simply deleting the image and their corresponding .txt files.

3. Training from Scratch vs Fine-tuning

Fine-tuning a pretrained model is the optimal approach for this dataset. With 15.7K training images, training from scratch would require the network to learn all visual representations from random initialization — a process that typically demands hundreds of thousands of images to converge effectively and carries significant overfitting risk at this scale. Fine-tuning leverages backbone weights pretrained on large-scale datasets like COCO, enabling faster convergence, stronger generalization, and substantially lower training compute cost. Since airborne imagery falls well within the natural image domain these pretrained weights already encode, we can leverage this prior knowledge.

4. Choosing the foundation model to fine-tune:

We explore the following options :

- a. RT-DETR / DINO / Co-DETR (transformer detectors)

Transformer detectors are accurate but they're heavy. They need large GPU memory, long training times, and their inference involves attention computations that scale quadratically with token count. The deeper problem is deployment. RT-DETR and other transformer-based detectors excel in high-capacity GPU environments, but if we decide to deploy this model on edge devices, we can face challenges. Additionally, transformer detectors still typically require NMS or Hungarian matching as post-processing, adding pipeline complexity.

- b. Faster R-CNN / Cascade R-CNN

They are two-stage detectors which generate region proposals first, then classify them. Thus it's inherently 5-10x slower at inference compared to single-stage alternatives. For our use case, speed is critical. Moreover, they require manual anchor box configuration, meaning we need to carefully define anchor scales, aspect ratios, and strides that match our specific object sizes, and if we get these wrong our recall drops significantly, especially on small or unusually shaped airborne targets.

- c. YOLOv26 (You only look once : 1 stage process)

YOLO26 becomes the most practical choice because :

- It is lightweight by design — it removed DFL entirely to simplify the architecture and was purpose-built for edge and low-power devices where compute and memory are constrained.
- It solves the hardest part of our problem — detecting small objects — through its STAL and ProgLoss mechanisms, which actively prioritize learning from tiny, hard-to-see targets during training without any manual tuning.

- It also completely eliminate NMS post-processing, giving a deterministic, predictable latency at inference time, which matters for this use-case.
- It trains faster than previous YOLO versions due to MuSGD optimizer.

5. Deep Learning Architecture

YOLO26's architecture :

https://drive.google.com/file/d/1lsecPCKzILbMSfdgPmigO5E0p5Sc6D0j/view?usp=drive_link

Architecture is split into three main components:

- Backbone** — The feature extractor. It takes the raw 640×640×3 input image and progressively downsamples it through convolutions and C3k2 blocks, producing feature maps at multiple scales (P2 through P5). It learns what features exist in the image at different resolutions. Deeper layers capture more semantic (high-level) information, shallower layers capture fine spatial detail.
- Neck** — The feature fusion network. It follows a PAN/FPN-style design: it takes the multi-scale features from the backbone and fuses them together using upsampling, concatenation, and more C3k2 blocks. The purpose is to ensure that every detection scale has access to both fine-grained spatial detail (from early layers) and rich semantic information (from deep layers).
- Head** — The prediction module. It takes the fused features at three scales (P3, P4, P5 — corresponding to 80×80, 40×40, and 20×20 grids) and produces the final detections (bounding box coordinates + class predictions). A key YOLO26 innovation: it's a native end-to-end model that produces predictions directly without NMS Ultralytics, using a one-to-one detection head.

6. Fine-Tuning Approach:

We Fine-tune on the entire YOLOv26 Model (weights of all 3 components are updated during training) since we have more than 1000 annotations per class & our use case is of completely different domain (different object types, sizes, lighting conditions, and contexts).

7. Evaluation Metrics

Detailed Explanation of each metric:

https://docs.google.com/spreadsheets/d/1ZgEAvPyFTF7az3tMMlctliPcB_lF9iA/edit?gid=620564821#gid=620564821

7. Timeline of Project

https://docs.google.com/spreadsheets/d/1HeelJQXGQV3pywCgoPalnddk_Z6R5h2exwTtL8uuz-g/edit?usp=drive_link